

SDC 학생강사

컴퓨터공학 입문자를 위한 자료구조론

1주차

CONTENTS

01

Git / Github 기본 사용법의 이해

해당 목차에 대한 설명을 입력해 주세요

02

포인터 개념을 기반으로 Linked List 구조 이해

해당 목차에 대한 설명을 입력해 주세요

03

시간복잡도

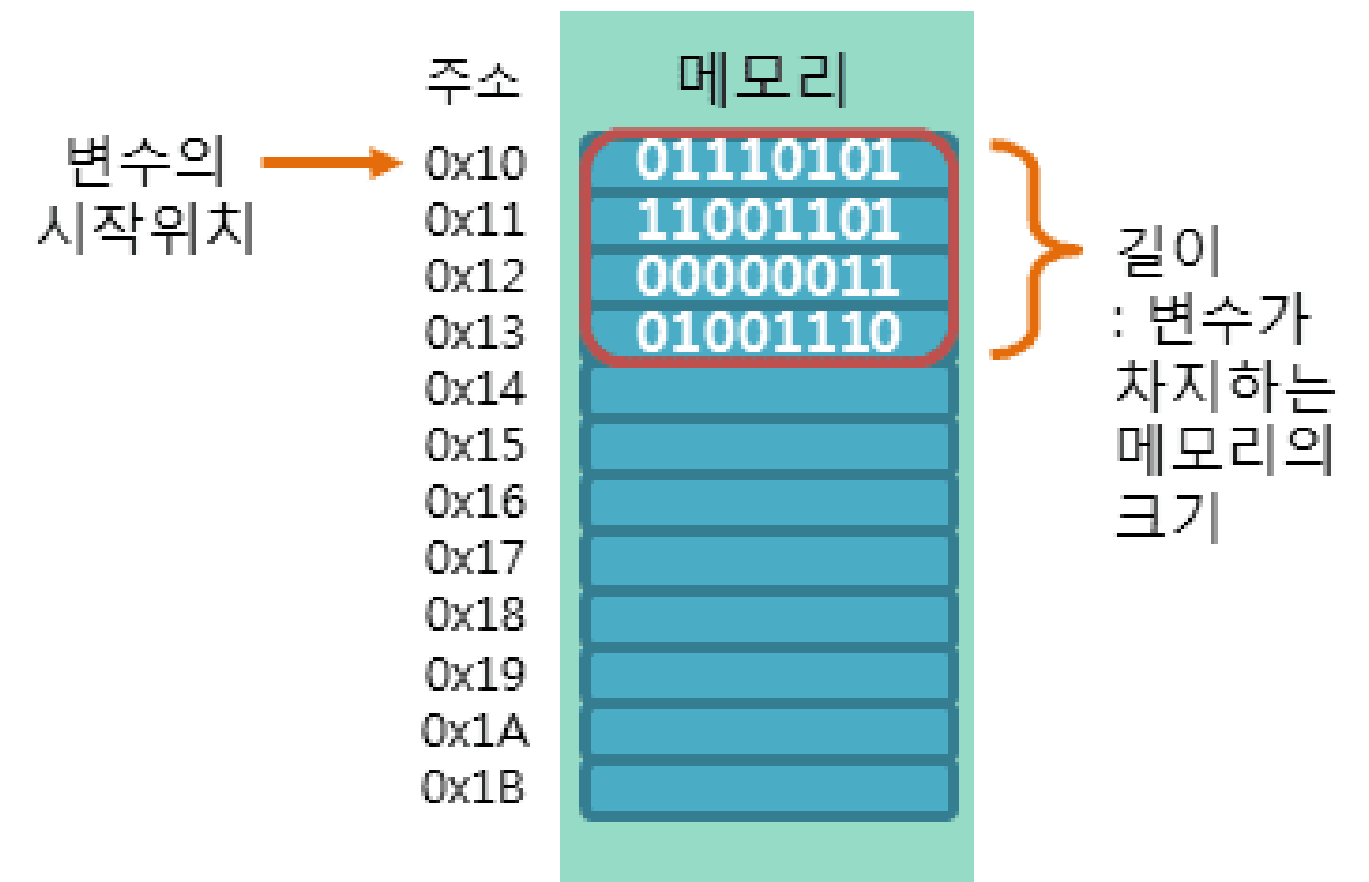
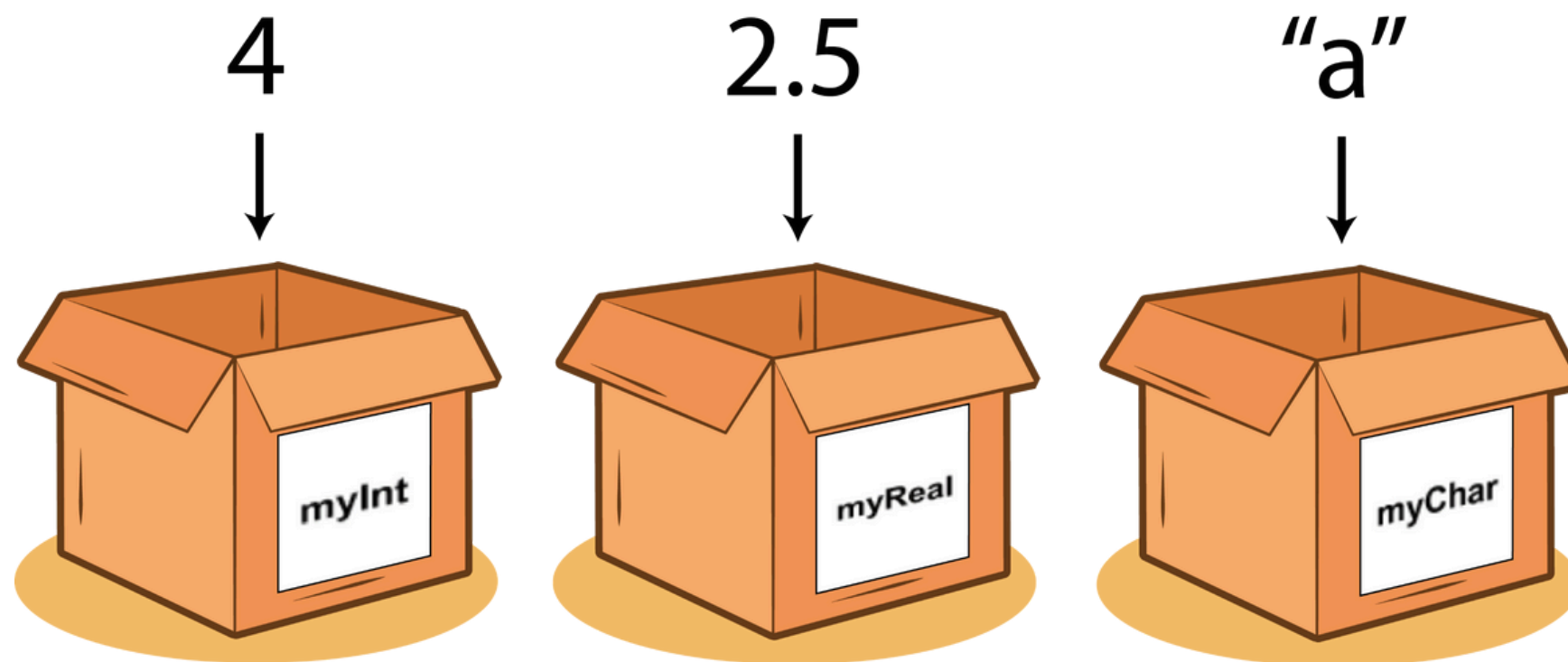
해당 목차에 대한 설명을 입력해 주세요

02

링크드 리스트와 포인터

변수란?
variable

변수를 선언하면 컴퓨터내의 메모리 공간을 확보해, 그곳에 데이터를 저장 할 수 있다.
또한 내가 선언한 변수의 이름을 통해 해당 메모리 공간에 저장한 데이터에 접근이 가능해진다.



형태 : 메모리에 저장된 데이터를 해석하는 방법

02

링크드 리스트와 포인터

포인터는?

변수는 메모리 공간에 데이터를 넣고 변수 이름으로 그 데이터 공간에 접근이 가능 하다고 했다

그럼 메모리 주소 자체를 저장하기 위해선?

메모리 주소 자체를 저장하는 행위가 왜 필요하지?

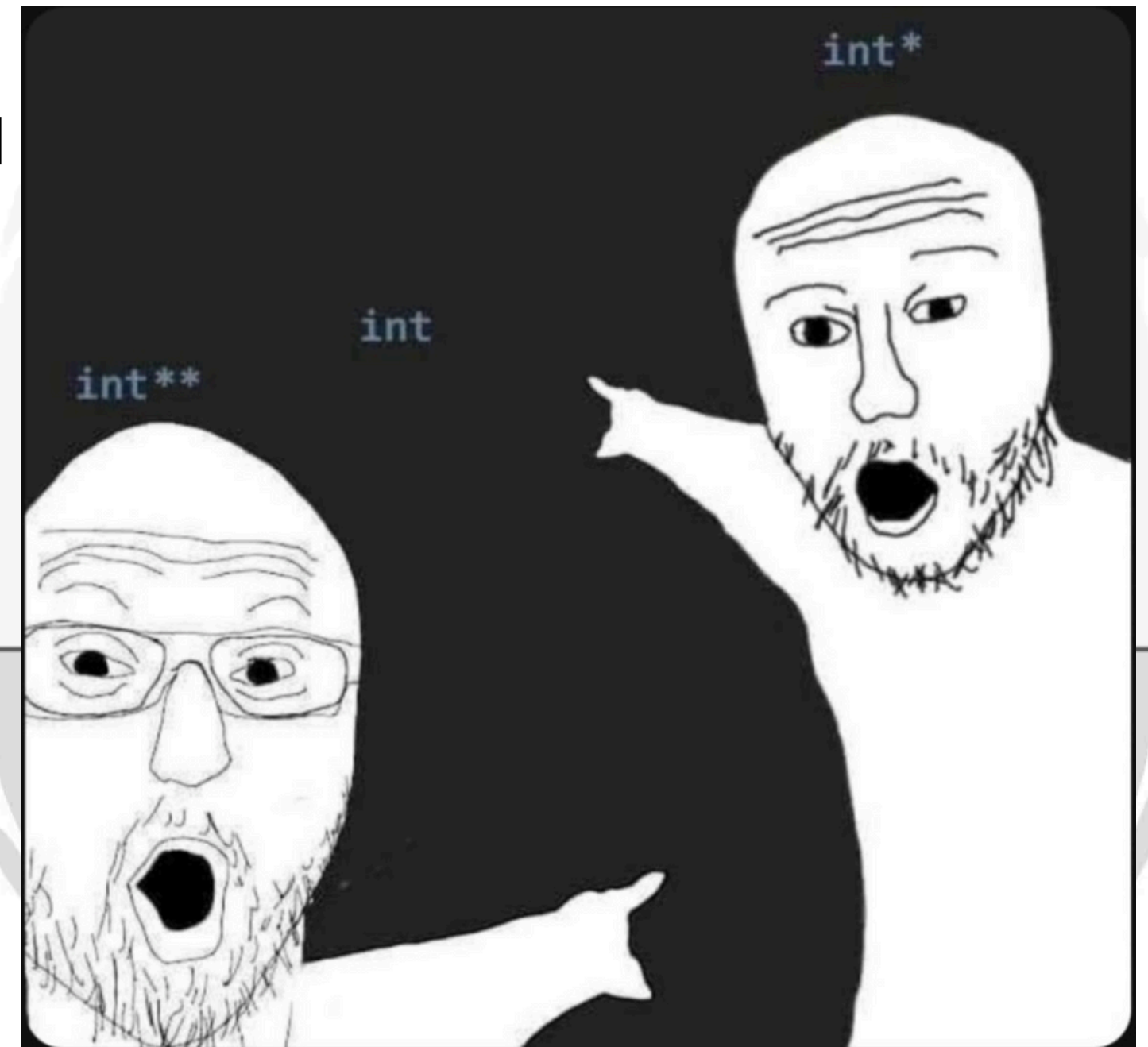
실용적인 이유

1. 함수 원본 데이터 변경
2. 동적 메모리 할당
3. 데용량 데이터의 전달

자료구조에서는?

포인터를 통해 메모리에서 흩어진 데이터를 논리적으로 관리
할 수 있음

논리적으로 추상화된 개념을 이해하는것이 컴퓨터 공학을
이해하는 첫 걸음



02

링크드 리스트와 포인터

포인터는?

주소 연산자

- 주소 연산자(&)는 변수의 주소를 반환한다.

```
int num;  
int *pi = &num;
```

정수를 가리키는 포인터 변수 pi는 변수 num의 주소를 담고 있다.

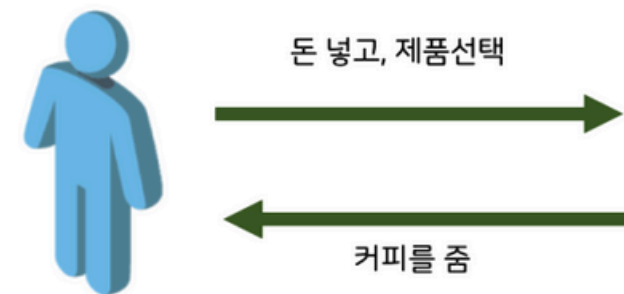
```
int num = 0;  
int *pi = num;
```


02

링크드 리스트와 포인터

ADT Abstract Data Type

ADT(추상 데이터 타입) 은 데이터의 논리적 모델로, 자료구조를 추상적(수학적) 으로 정의하며 해당 모델이 제공해야하는 기능만 정의한다,



제품선택 → 함수(메서드)
돈넣고 → 함수호출시 전달 파라미터)
커피를 줌 → 리턴값



우리는 자판기를 사용할때
버튼을 누르면 → 음료를 준다는것 만
알면되고 자판기 내부 원리를 이해 할 필요는 없다

추상화, 인터페이스

우리 수업에서는?
필요한 기능명세를 바탕으로 적절한 내부 구현을 직접 해보자

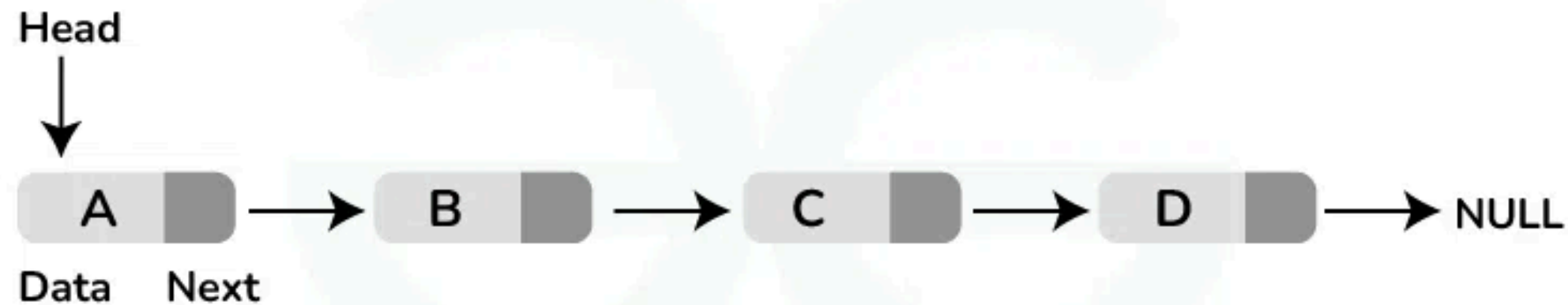
ADT를 통한 자료구조 구현

02

링크드 리스트와 포인터

단일 연결 리스트
Linked List

Data, Next 두가지 부분으로 이루어진 Node를 기준으로
하나의 Node가 다음 Node를 가르키는 구조가 반복되어 나타나는 자료구조



Singly Linked List



02

링크드 리스트와 포인터

Linked List의 ADT

추가(Add)

tail 뒤에 새로운 노드를 추가

삽입(Insert)

Node 중간에 새로운 Node 삽입

삭제(Remove)

Node 제거

탐색(Search)

원하는 Node 찾기

02

링크드 리스트와 포인터

Linked List

```
#include <iostream>
#include <string>
using namespace std;

typedef string Elem;

class StringNode {
private:
    Elem elem;
    StringNode* next;

    friend class StringLinkedList;
};

class StringLinkedList {
private:
    StringNode* head;
    StringNode* tail;
public:
    StringLinkedList();
    ~StringLinkedList();
    bool empty() const;
    Elem front() const;
    void addFront(const Elem& e);
    void removeFront();
    void addBack(const Elem& e);
    void removeBack();
};
```

수업에서는 C++를 사용해 구현 할 예정이기에
class로 다음과 같이 구현 할 수 있다.

02

링크드 리스트와 포인터

배열과 링크드 리스트

C, C++에는 기본적으로 배열이라는 구조가 존재하는데 굳이 왜 이렇게 값을 저장해야하지?

배열은

연속된 메모리 공간을 먼저 할당하고, 저장한다.
따라서 생성시 크기를 미리 정하고 바꿀 수 없다

반면 링크드 리스트는

Node들이 메모리에 임의대로 흩어져있고, 연속되지 않는다.
하지만 Pointer를 활용해 크기를 동적으로 늘이고, 줄일 수 있다.

추가로

가장 기본이 되는 링크드 리스트를 활용해 추후에 공부할
여러 자료구조를 편하게 구현 할 수 있다.
(물론 배열이 더 편한 경우도 존재함)

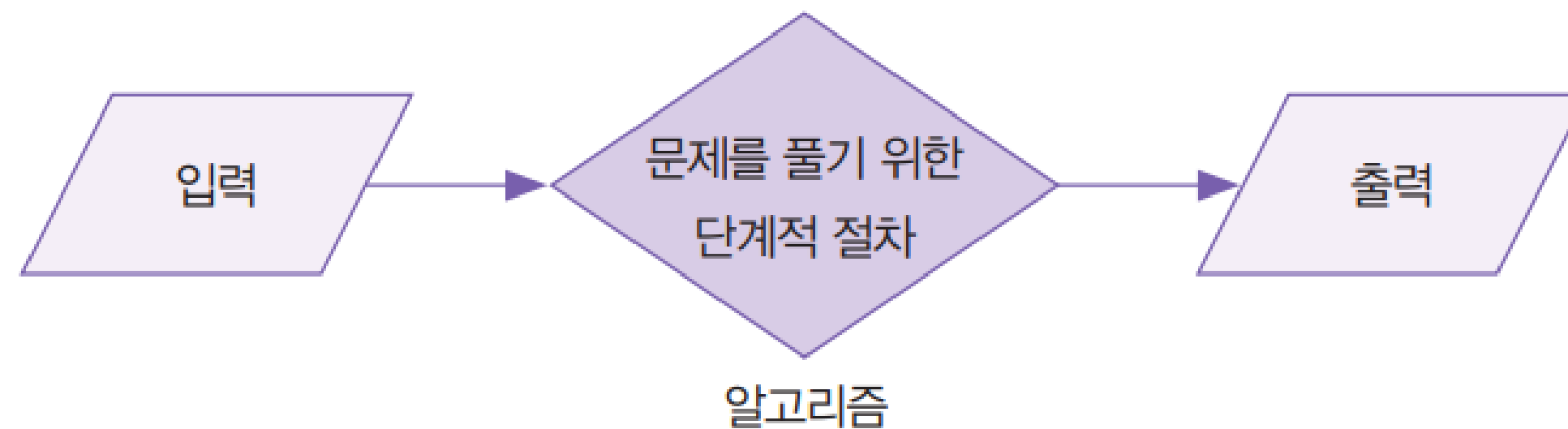
기준	배열(Array)	링크드 리스트(Linked List)
메모리 배치	연속된 공간에 저장 1	비연속 공간, 노드 포인터로 연결 2
크기	생성 시 고정, 변경 어려움 1	동적 증가/감소 용이 2
조회 성능	인덱스 기반 임의 접근 $O(1)$ 1 7	순차 접근 $O(n)$ 1 6
중간 삽입/삭제	요소 이동 필요, 보통 $O(n)$ 1 7	포인터 변경은 $O(1)$, 위치 탐색은 $O(n)$ 1 6
메모리 효율	포인터 오버헤드 없음, 비교적 효율적 1 6	포인터 저장 때문에 오버헤드 존재 1 3
적합한 상황	크기가 거의 고정이고 조회가 잦을 때 1 6	크기 변화·중간 삽입/삭제가 잦을 때 1 6

03

시간 복잡도 분석

알고리즘이란?

어떤 문제를 풀기 위한 일련의 논리적 절차, 명령어의 집합
프로그래밍에서는 입력과 출력으로 나누어 생각 할 수 있다.



자료구조는 데이터를 효율적으로 저장,관리, 접근하기 위한 뼈대
알고리즘은 자료구조를 기반으로 문제를 해결하는 방법, 절차

라고 생각 하면 쉽다

따라서 빠르게 문제를 해결하기 위해서 얼마나 효율적이고 빠른 자료구조를 선택하고, 활용하는지가 관건이다.

03

시간 복잡도 분석

실험적 분석 - Experimental studies

실제로 코드를 동작시켜 실행시간을 분석하는 방법

1. Average case

100개의 케이스를 다 측정한다음 모두 합해 100으로 나누는 방법이다.
이는 잘 조정하면 의미있는 데이터가 되겠지만, 일반적인 정의를 내리긴 힘들다.

2. Worst Case

모든 케이스중 가장 오래걸리는 시간을 측정한다.
이는 구하기 쉽고, 기능과 직접적으로 연관되어있다.
이러한 측정은 clock() 같은 함수를 사용하여 정확한 시간을 측정한다.

한계점

- 1) 알고리즘을 실행하기 위해 오류가 없어야 한다.
- 2) 모든 인풋 사이즈 케이스에 대한 수행시간이 아니므로 일반화의 오류가 생김
- 3) 컴퓨터의 성능(hw, sw환경)에 의해 실행시간이 달라진다

03

시간 복잡도 분석

이론적 분석 - Theoretical Analysis

Input case N에 대한 함수로 실행시간을 이론적으로 분석

1. Psuedo code

자연어를 이용해 프로그래밍 언어를 유사한 형식으로 배치한 코드

알고리즘이 수행될 내용을 알기 쉽게 간략히 설명함

Pseudo code of Static $P\&Q$ Rule

```
Repeat
  Compute  $EMA(T)$ 
  If no position opened
    If  $EMA(T) \geq P$ 
      If trend is going up
        Open a long position
      Else if trend is going down
        Open a short position
    Else if any position is opened
      If  $EMA(\neg T) \geq Q$ 
        Close position
  If end of market
    Close all opened position
Until market close
```


03

시간 복잡도 분석

시간복잡도

입력데이터가 커질 수록 연산의 실행횟수가 커지므로
연산의 실행횟수를 입력의 크기에 대한 함수로 표현한다

실행 횟수 계산을 수도코드를 참조하여 분석한다

※ 유사코드 (Pseudo-Code) 로 표현한 알고리즘

반복문의 연산은 카운트하지 않는다.

```
result ← n * n
```

대입연산

알고리즘 A

```
for i ← 1 to n do  
  result ← n * n
```

알고리즘 B

```
for i ← 1 to n do  
  for j ← 1 to n do  
    result ← n * n
```

알고리즘 C

※ 각 알고리즘의 연산 비교

연산	알고리즘 A	알고리즘 B	알고리즘 C
곱셈연산	1	n	n^2
대입연산	1	n	n^2
연산의 합	2	2n	$2n^2$

03

시간 복잡도 분석

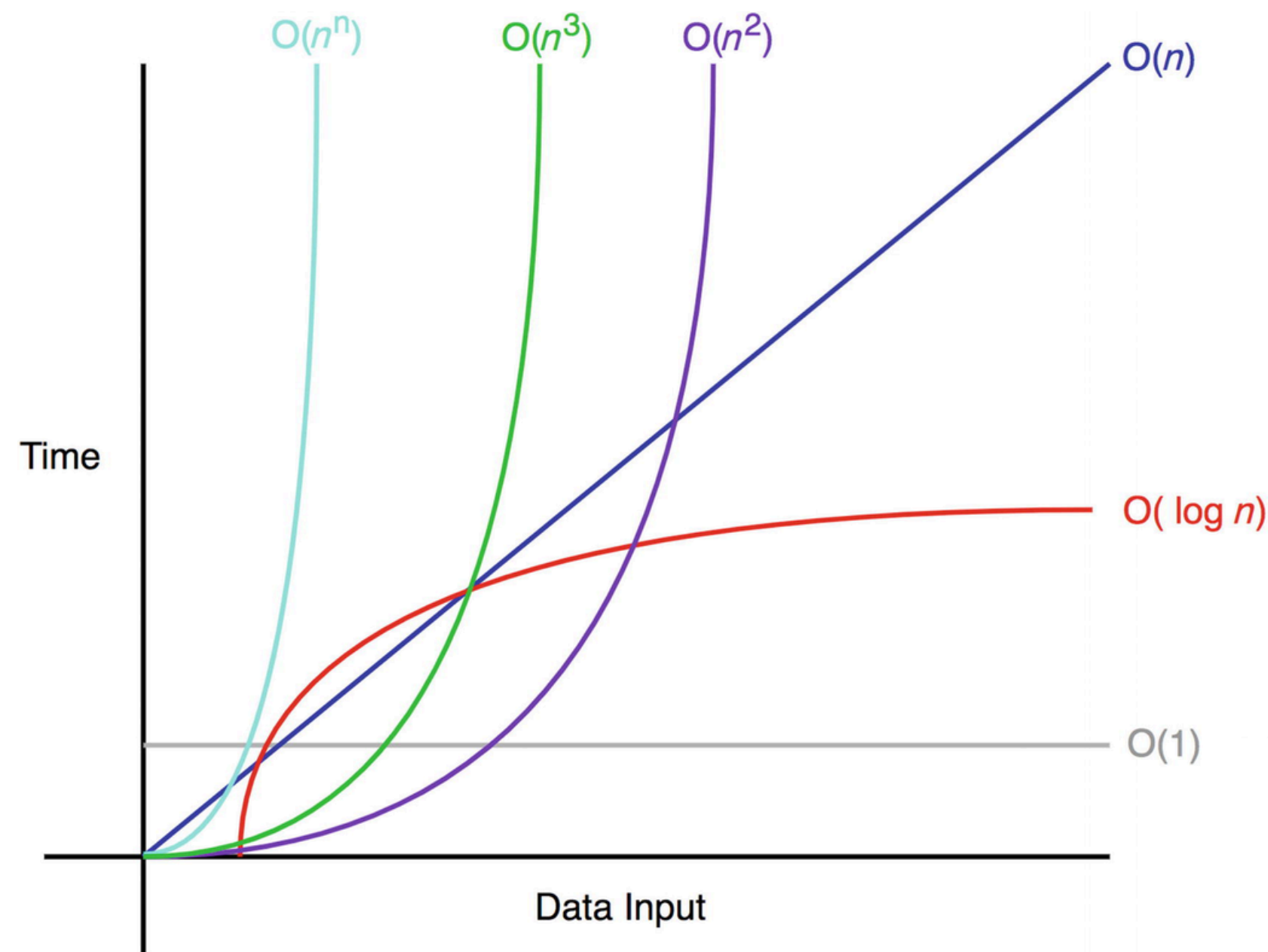
빅오 표기법(Big-O)

함수의 증가율을 수학적으로 표기한것

점근적 분석

입력의 크기가 무한히 커질때 알고리즘의 효율적을 판단하는 방법

- 상수 계수와 낮은 차수 항목은 무시합니다.
 - 입력 크기가 충분히 클 때의 증가율에 주목합니다.
 - 알고리즘의 확장성(scalability)을 평가합니다.
-
- 빅오(O): "최악의 경우에도 이것보다 느리지 않아요"
 - 빅오메가(Ω): "최선의 경우에도 이것보다 빠르지 않아요"
 - 빅세타(Θ): "항상 이 정도 시간이 걸려요"



04

과제

1. 레포지토리 fork 하기
2. 개인 깃허브에 README 생성
3. 이 수업에서 배우고 싶은것/ 얻어가고 싶은것 작성해서 COMMIT 하기